

HANOI UNIVERSITY OF SCIENCE AND
TECHNOLOGY

School of Information and communications technology

DESIGN PRINCIPLES REPORT

AIMS Project

Subject: ITSS Software Development

Group 18

Nguyễn Tuấn Đạt – 20235907

Thành Hựu Đạt – 20235909

Lê Tùng Lâm – 20235962

Chu Tuấn Linh – 20235963

Tạ Hải Luân – 20235969

Hanoi, May 2026

1. Single Responsibility Principle (SRP)

Module	PIC	Description	Impact	Improvement Direction
ProductService	Linh	This service handles multiple responsibilities in product CUD: data validation, product creation/update, type-specific detail synchronization, delete/deactivate handling based on stock, delete-count limitation, and history logging.	A small change in validation rules, history logging, or product subtype behavior may require changes in multiple parts of the same service. Unit tests are also harder to isolate.	Split it into ProductCommandService, ProductTypeDetailService, ProductDeletionPolicy, ProductValidator, and ProductHistoryService .
ProductListCreateView	Linh, Lam	This view combines customer product listing, manager product listing, and manager product creation.	Customer read flow and manager write flow have different reasons to change but are currently placed in the same controller.	Split customer read views from manager command/list views. Selectors/services may still be shared where appropriate.
ProductForm	Linh	This component manages form state, price hints, subtype resets, payload shaping, create/edit mode, and submit status.	When form rules, subtype fields, or layout change, the component can become large and difficult to test.	Move form state/validation into a hook such as useProductForm; move subtype behavior into a registry or dedicated components.

ProductListScreen	Lam	This component loads products, manages filters/pagination, handles add-to-cart errors, and opens the detail popup.	Product browsing and cart interaction are coupled in the same screen, making UI and flow tests more complex.	Extract useProductCatalog and useAddToCart hooks; keep the screen focused on composing layout.
ProductDetailPopup	Lam	This popup fetches product details, manages loading/error state, quantity state, add-to-cart action, and modal rendering.	Data loading, cart actions, and modal UI may change independently but are currently placed in one component.	Move fetch and cart actions into hooks; let the popup receive a view model and focus on rendering.
ProductDeleteDialog	Linh	This dialog renders confirmation UI but also contains text describing backend delete/deactivate rules.	If backend rules change, the UI copy may become inconsistent with actual behavior.	Pass policy text from the parent component or API metadata; keep the dialog focused on confirming the action.
CartService	Dat Thanh	This service combines cart retrieval/mutation, stock validation coordination, subtotal/weight calculation, and ORM access.	Cart mutation, checkout validation, and summary calculation have different reasons to change but are placed in the same service.	Split cart command handling, stock policy, totals calculator, and repository/query helpers.
CartSerializer	Dat Thanh	This serializer formats the response but calls CartService to calculate subtotal, canPlaceOrder, and stockErrors.	Serialization becomes dependent on business validation, so API output tests depend on service logic.	Return a precomputed read DTO from the service or introduce a cart summary read model.

CartScreen	Dat Tha nh	This screen loads the cart, updates/removes items, creates a draft order, stores checkout tokens, and navigates to checkout.	Cart UI and order-start workflow are combined, making them harder to test independently.	Extract useCart and useStartCheckout hooks.
carts/api.ts	Dat Tha nh	This API module manages cart token storage, HTTP calls, and cart-count event broadcasting.	Storage, networking, and UI synchronization are coupled.	Extract token storage and cart event publishing from the API client.
OrderPlacementService	Dat Tha nh	This service coordinates draft orders, delivery information, invoices, stock validation, order confirmation, paid-state marking, stock mutation, notifications, and refund triggering.	This is a large orchestration service with many reasons to change: checkout, payment success, cancellation, stock, and email.	Keep it as an orchestration service but inject collaborators for delivery, invoice, stock, notification, and refund/payment boundaries.
validate_delivery_info / DeliveryInfoValidator	Dat Tha nh	This validator combines many delivery field rules and switches behavior using require_full.	When new delivery modes or optional flows are added, the validator can accumulate more branching logic.	Split field validators or validation rule objects by scenario.
OrderDraftView and order checkout views	Dat Tha nh	These order views repeat request parsing, exception mapping, and service delegation.	Changes to error handling may require editing multiple controllers.	Create a shared base helper or API adapter for exception mapping; keep each view focused on one HTTP action.

OrderCancelView	Dat Tha nh	This view directly looks up orders by cancel token before delegating cancellation.	Public-token lookup and cancellation policy are hard to test separately and may be duplicated in other public endpoints.	Move token lookup into a selector or service.
DeliveryInfoForm	Dat Tha nh	This component manages form state, error mapping, live delivery preview, submit behavior, and navigation.	Delivery UX and fee-preview behavior are difficult to change independently.	Extract useDeliveryForm and useDeliveryPreview hooks.
InvoiceScreen	Dat Tha nh	This screen loads the invoice, renders item/delivery details, confirms the order, and navigates to payment.	Invoice presentation and checkout orchestration are coupled.	Move invoice loading and order confirmation into a hook.
InvoiceSummary	Dat Tha nh	This component is currently display-only; the comment warns against adding payment/navigation logic here.	If payment logic is added to the summary, the component will lose reusability.	Continue passing precomputed invoice totals; keep payment actions in the parent component.
orders/api.ts	Dat Tha nh	This API module combines draft order, delivery, invoice, mark-paid, tracking, cancellation, and preview endpoints.	Checkout APIs and public tracking/payment-transition APIs change for different reasons but are placed in the same file.	Split it into checkoutApi, orderTrackingApi, and orderPaymentApi.

VietQRService	Luan	This service creates transactions, calls the gateway, validates callbacks, updates payment state, and attempts to synchronize order status.	The QR workflow is coupled to persistence, gateway construction, and order integration.	Inject gateway and order-fulfillment collaborators; separate callback validation from transaction persistence.
VietQRGateway	Luan	This gateway handles token retrieval, settings, HTTP calls, response parsing, QR data URL conversion, and ORM-based status lookup.	Gateway tests require both configuration and database state; status or token changes may affect QR generation.	Inject token, configuration, and status collaborators; keep the gateway focused on mapping the HTTP provider to DTOs.
VietQRTransactionSyncView	Luan	This view verifies bearer tokens, parses provider payloads, calls the service, and formats the response envelope for VietQR.	Callback authentication, provider mapping, and payment confirmation are coupled inside the controller.	Move callback authentication and payload mapping into a handler/service.
QRPaymentScreen	Luan	This component creates QR payments, checks status, performs mock confirmation, marks the order as paid, and handles routing.	The UI depends directly on the backend payment workflow, making it harder to test or replace the provider.	Move QR orchestration into a hook/service; keep the component focused on rendering QR/status/action UI.
payments/api.ts	Luan	This API module mixes production VietQR endpoints, mock callbacks, and error translation.	Demo/mock changes may affect the production API module.	Move mock/demo helpers out of the production payment API.

PayPalGateway	Đạt Tuần	This gateway combines order creation, capture, refund, OAuth token caching, mock mode, and HTTP error mapping.	Authentication, transport, mock/demo logic, and provider mapping change within the same class.	Extract token manager and mock adapter; keep the gateway focused on PayPal REST mapping.
PayPalPaymentView	Đạt Tuần	This view validates input, calculates amount/currency, calls the payment service, and creates transactions.	The controller mixes HTTP concerns, orchestration, and persistence.	Move transaction creation and amount selection into a PayPal application service.
PayPalPaymentButton	Đạt Tuần	This component loads the PayPal SDK, calls initiate/capture APIs, handles redirects, and renders fallback UI.	The UI is coupled to the PayPal global SDK and concrete backend APIs.	Move SDK loading and payment orchestration into a hook/service; inject a payment client.
PayPalPaymentInner	Đạt Tuần	This page parses query parameters, builds return/cancel URLs, captures payment, maps results, and renders multiple page states.	Routing, payment orchestration, and result presentation are coupled.	Move query parsing and capture orchestration into a hook/view model.
PaymentService	Luan, Đạt Tuần	This service exposes initiate, capture, and refund operations in the same service.	A caller that only needs capture still depends on refund behavior; the service can grow too large as operations expand.	Split payment orchestration from refund orchestration.

RefundService	Đạt Tuấn	This service focuses on refunds but depends on the overly broad PaymentGateway abstraction.	Refund-only logic is forced to depend on create/capture capabilities.	Use a smaller interface such as RefundProcessor.
---------------	----------	---	---	--

2. Open/Closed Principle (OCP)

Module	PIC	Description	Impact	Improvement Direction
serialize_type_details	Lam, Linh	The function hard-codes the ProductType -> related_name mapping.	Adding a new product type requires modifying the existing serializer.	Use a subtype serializer registry or a strategy per product type.
ProductSelector.list_for_customer	Lam	This method hard-codes filter/sort policies and random landing behavior.	Adding a new filter or sort option requires modifying the central method.	Extract filter/sort policies or use a query builder registry.
ProductTypeFields.FIELD_CONFIG	Linh, Lam	The field config hard-codes all subtype fields in the frontend.	Adding or changing a subtype requires modifying the component module.	Use a shared field registry or metadata from the backend.

TypeSpecificDetail	Lam	The component renders subtypes using switch(product.product_type).	Adding a new product type requires modifying the component.	Use a component registry keyed by product_type.
ProductForm	Linh	The form contains subtype switching and field reset logic.	Adding a new subtype can require changes inside the form.	Use a subtype behavior registry.
DeliveryFeeService / calculate_delivery_fee	Dat Thanh	The fee formula hard-codes city groups, base fee, weight steps, and discounts.	Adding a new shipping region or method requires modifying the function.	Use delivery fee policy objects or a configurable rate table.
determine_refund_action	Dat Thanh, Luan, Đạt Tuấn	The function hard-codes refund decisions for PayPal/VietQR based on payment_method.	Adding a new payment provider requires modifying the function.	Use a provider-specific refund policy registry.

PaymentTransaction	Luan, Đạt Tuấn	The model is gateway-agnostic but contains provider-specific fields such as capture_id, refund_id, transaction_reference, and provider_payload.	A new provider may require additional nullable or overloaded fields, making the model harder to extend cleanly.	Keep common fields in the base transaction and move provider metadata to extension records or typed DTOs.
PaymentResult	Đạt Tuấn	The local STATUS_CONFIG hard-codes result states.	Adding a new status requires modifying the component.	Pass display config from the caller or use a result-state registry.
products/api.ts	Linh, Lam	This file combines manager CUD endpoints and customer read/detail endpoints.	Extending manager or customer APIs makes the file harder to keep within a clear boundary.	Split it into productManagementApi and customerProductApi.

3. Liskov Substitution Principle (LSP)

Module	PIC	Description	Impact	Improvement Direction
--------	-----	-------------	--------	-----------------------

PaymentGateway	Luan , Đạt Tuấn	The abstract gateway forces every provider to implement create_payment, capture_payment, and refund_payment, even though QR/manual refund providers may not have the same capabilities as PayPal.	A provider may need to implement unsupported methods, making provider substitution fragile.	Split the interface by capability: PaymentCreator, PaymentCapturer, RefundProcessor, and StatusProvider.
PaymentTransaction	Luan , Đạt Tuấn	A single transaction model is shared, but provider-specific fields do not apply equally to every gateway.	Shared code may assume fields exist for providers that do not support them, making substitution between QR/card transactions fragile.	Standardize the common transaction contract and move provider-specific details into typed extension/metadata records.

4. Interface Segregation Principle (ISP)

Module	PIC	Description	Impact	Improvement Direction
PaymentGateway	Luan , Đạt Tuấn	The interface is too broad because it combines create, capture, and refund operations.	Gateway implementations depend on methods they may not need or support.	Split it into smaller capability-specific interfaces.
PaymentService	Luan , Đạt Tuấn	The service API combines initiation, capture, and refund orchestration.	A client that only needs one operation still depends on the whole service.	Split it into PaymentInitiationService, PaymentCaptureService, and RefundApplicationService if the flow continues to grow.

RefundService	Đạt Tuấn	RefundService depends on PaymentGateway instead of a refund-only interface.	The refund module is coupled to unused create/capture methods.	Create a RefundProcessor abstraction with only refund_payment.
---------------	----------	---	--	--

5. Dependency Inversion Principle (DIP)

Module	PIC	Description	Impact	Improvement Direction
ProductService	Linh	This service directly calls ORM models and ProductHistoryService.	Product CUD unit tests are harder because persistence and history logging are difficult to mock.	Inject repository/history abstractions or service collaborators.
CartService	Dat Thanh	This service directly depends on Product, Cart, and CartItem ORM models.	Cart behavior is hard to test without a database.	Extract a repository/query layer or use injected collaborators for stock lookup.
CartSerializer	Dat Thanh	This serializer directly depends on CartService to calculate business output.	API presentation depends on a concrete service implementation.	Pass a summary DTO from the service/controller into serializer context or use a read model.
OrderPlacementService	Dat Thanh	The refund path directly imports and instantiates PayPalGateway, RefundService, and payment models inside the order service.	The order module is coupled to concrete payment implementations.	Inject payment/refund abstractions or use domain events so the payment module handles refunds.

_get_payment_service	Đạt Tuấn	The view factory directly constructs PayPalGateway and RefundService.	View tests and provider replacement depend on concrete PayPal configuration.	Move construction into a provider registry, factory, or DI boundary.
VietQRService	Luan	The constructor directly creates VietQRGateway by default, and the service also handles order synchronization.	Tests must manually mock concrete gateway and order behavior.	Inject the gateway and an order-fulfillment collaborator.
VietQRGateway	Luan	This gateway directly depends on settings, token manager, and ORM status lookup.	Token, configuration, or status implementations are hard to replace.	Inject configuration, token, and status providers.
QRPaymentScreen	Luan	This component directly imports API functions and markOrderPaid.	The UI depends on the concrete backend workflow.	Inject or use a payment-client hook.
PayPalPaymentButton	Đạt Tuấn	This component directly depends on the PayPal global SDK and concrete payment API functions.	The SDK/API are difficult to mock in tests, and replacing the hosted provider is harder.	Extract a PayPal SDK client and payment client abstraction.